

Motion-Painter — Canonical Development Guide (vNext)

Motion-Painter Team — vNext Comprehensive Proposal & Implementation Guide

Generated by assistant

This PDF is the canonical development guide for Motion-Painter. Use it as the authoritative reference for implementation.

Overview

This document is the canonical development guide for Motion-Painter. It brings the current implementation up to date and describes proposed next-generation subsystems: hybrid render policy, motion-worker, MultiSampler integration, Hybrid Fresnel → Pascal → Minimizer pipeline, Vault architecture, Lipschitz quaternion motion maps, Prime-Ends topology sampling, and other supporting modules. The goal is to provide an authoritative reference for contributors and to produce an implementation roadmap with detailed, actionable subtasks and API contracts.

Current System (brief)

Motion-Painter core currently provides:

- WebGL2 renderer with array-texture ring buffer for historical frames.
- FrameBuffer (circular) managing GPU textures and upload pipeline.
- Preprocessor worker that ingests ImageBitmap frames, computes thumbnails, phash, calibration, and writes artifacts to storage via storage.js. Calibration (dark/flat/bias) subsystem exists.
- Composite and motion shaders (motion.frag) that compute luminance motion and visualizations.
- MotionDetector.js — a CPU analysis helper exposing `getMotionParams()` and `analyzeMotion()`.
- UI, MediaInput and Controls, and a storage/vault subsystem skeleton. The codebase exposes MotionPainter on window for diagnostics and runtime hooks (evictionHook, preprocessor, etc.).

Design Goals for vNext

1. Decouple rendering cadence from data processing while maintaining perceived smooth frame rate.
2. Create a dedicated motion-worker that offloads motion analytics and sampling from the main thread and from the preprocessor worker to avoid contention.
3. Implement a hybrid adaptive render policy (Markov + inverse Polya urn + temporal interpolation + motion-aware sampling) that adapts in real time to frame eviction and buffer state.
4. Integrate MultiSampler (Wallis/Random/Vogel) as core sampling engine used by the motion worker and later by the renderer for primitive generation.
5. Add Hybrid Fresnel → Pascal → Minimizer pipeline to produce geometric primitives and priors.
6. Provide Vault architecture for long-term artifact retention and metadata indexing, separate from transient storage used by preprocessor.
7. Expose telemetry, diagnostics and tuning UI for parameter exploration and dynamic equilibrium.
8. Keep updates modular and backward compatible with current Motion-Painter code paths.

High-level Architecture (vNext)

The updated architecture organizes responsibilities into clearly separated subsystems and well defined message contracts. Core components (existing + new):

- Main Thread (MotionPainter controller) - Orchestrates render loop, owns WebGLRenderer & FrameBuffer, UI, and worker lifecycle. - Receives lightweight motion hints and sampler manifests from motion-worker and decides what to render. - Publishes events/commands to workers and storage via BroadcastChannel or postMessage.
- FrameBuffer (GPU Ring Buffer) - Continues to manage texture array / arrayTexture approach. - Exposes metadata: frameCount, bufferSize, spiralIndices, upload timestamps, layer occupancy. - Emits hooks for eviction; evictionHook controls which frames are forwarded to preprocessor.
- Preprocessor Worker (existing) - Unchanged responsibilities: create thumbnails, phash, calibration, write artifacts into storage. - Will be extended to publish camera/track fields and lightweight analysis telemetry (see changes).
- Motion-Worker (new) - Dedicated worker for motion analysis and sampling. - Accepts small thumbnails or tiny ArrayBuffer pixel snapshots (ImageBitmap transfer is ideal). - Runs MultiSampler, Prime-Ends/topology plugins, Lipschitz quaternion mapping, and returns

sample manifests and motion descriptors to main thread for rendering decisions. - Does NOT own storage artifacts; instead it consumes thumbnails (transferred) or keys for stored thumbnails. - Optionally writes specialized artifacts (motion masks, sample manifests) to Vault via storage API.

- **Renderer (main thread + GPU shaders)** - Receives sampling manifests (points, weights, motion masks) and produces a smooth render stream. - Uses temporal interpolation, motion-aware stride, and Markov policy to select layers to draw while maintaining target display framerate. - Uses shaders for interpolation/flow, selective blending, and optional GPU motion mask computation.

- **Vault & Storage (storage.js)** - Storage retains artifacts for longer lifetimes and higher durability; Vault has its own retention policies, indexes and metadata schema for primitives and priors (Fresnel outputs, Pascal blobs, minimizer results, calibration meta, thumbnails, phash, pose, SDFs, etc.). - Preprocessor writes canonical thumbnails/manifest/phas; motion-worker may request read handle to Vault artifacts.

- **Telemetry/Control Panel (UI)** - Lightweight, collapsible diagnostics panel to tune sampler weights, motion thresholds, render stride, Markov temperature and Polya urn parameters. Exposes aggregated metrics and time series.

Message paths: Camera → FrameBuffer → (1) WebGLUpload → Render selection, (2) Eviction Preprocessor → Storage/Vault. Motion-worker receives transferred thumbnails (or reading from persisted thumbnails) and returns sampler manifests to Main.

Motion-Worker — Spec & API

Purpose: Fast motion analysis, sampling, topology extraction and producing concise manifests for the renderer. Design principles: • Lightweight interface with explicit ownership of transferred ImageBitmaps. • Deterministic, time-budgeted processing with cancel tokens. • Plugin architecture: register MultiSampler, topology plugin, quaternion motion mappers, and custom analyzers. • Telemetry and cost accounting per job (processingTime, queueUtilization, dropRate).

API (worker messages):

1. **init** { op: 'init', config: {...} }

2. **analyzeFrame** // small payload; prefer ImageBitmap transfer for thumbnails or ArrayBuffer of small size { op: 'analyzeFrame', jobId, imageBitmap?, thumbKey?, timestamp, frameNumber, meta } Response: postMessage({ event: 'analysis:ready', jobId, manifest, metrics })

3. **samplePoints** { op: 'samplePoints', jobId, input: ImageBitmap|ArrayBuffer|thumbKey, options } Response: { event: 'sample:ready', jobId, manifest: { samplePoints, weights, map }, metrics }

4. **fetchPersistedThumbnail** { op: 'fetchThumb', jobId, thumbKey } Response: { event: 'thumb:fetched', jobId, thumbBitmap (transferred), meta }

5. **shutdown / health-check / config update**

Processing Notes: • MultiSampler should be instantiated in worker; use createLightweight for per-frame inline budgets. • For heavy operations (topology, Lipschitz quaternion transforms), use dedicated job priorities and limit concurrency. • Always close ImageBitmaps after use; transfer ownership semantics must be explicit between main and worker. • Motion-worker should not write canonical artifacts to Vault unless explicitly configured to persist sample manifests or masks.

Motion Detector vs Motion-Worker

Role Clarification: • MotionDetector.js (existing) - is a synchronous, small CPU helper used by shaders and by main thread for fast parameterization. - Exposes `getMotionParams()` and `analyzeMotion(currentImageData, previousImageData)` for coarse, bounded analysis. - Lightweight and cheap — suitable for immediate render-time gating decisions.

• Motion-Worker (new) - performs heavier, batched, time-budgeted analysis (topology, phase comparisons, variance maps, prime-ends analysis, Lipschitz quaternion calculations). - Consumes thumbnails or small bitmaps from the preprocessor or main thread. - Produces sampling manifests, motion masks, skeleton priors and other descriptors for the renderer and downstream processors.

Integration patterns: • Fast path: Main thread uses MotionDetector for immediate shader uniforms and render gating. • Mid/long path: Motion-Worker provides richer manifests asynchronously to refine rendering strategy, update MultiSampler weights, and store manifests in Vault. • Preprocessor/Storage: Preprocessor continues to create thumbnails and writes them to storage; motion-worker may either receive thumbnails transferred directly from main/preprocessor or fetch persisted thumbnails by key.

Hybrid Adaptive Render Policy — Design

Goal: Keep visuals smooth (stable display FPS) while ensuring downstream data processing receives sufficient frames. The policy adaptively selects which buffer layers to sample and render based on buffer state, eviction behavior, and motion manifests.

Key concepts and signals: • `displayTargetFps` — the visual frame rate aimed for (e.g., 60/30). • `uploadRate` — actual incoming frame rate from camera (~30fps). • `bufferSize` & `fillRatio` — how many frames held; eviction pressure and spillover. • `evictionRate` — how many frames per time unit are being evicted to preprocessor/storage. • `motionIntensity` — aggregated motion level from motion-worker / MotionDetector. • `temporalCoherenceScore` — similarity of adjacent frames where interpolation is feasible. • `stride` — rendering step across buffer layers (e.g., render every Nth frame) — adaptive.

Hybrid policy components: 1. Markov short-term policy - Model per-frame selection as a simple Markov chain where state = current writeIndex/layer. - Transition probabilities favor recent frames but allow occasional skips to reduce upload contention. - Transition matrix can be temperature-controlled based on motionIntensity (higher motion -> favor current frames).

2. Inverse Polya urn penalty - Maintain soft counts of recent selections for layers/regions; inverse Polya reduces probability of repeating same exact layer/region to avoid overpainting and visual staleness. - Acts as an anti-repetition regularizer; stronger when motion is low.

3. Temporal interpolation + motion-aware sampling - Use motion masks to decide where to interpolate between frames vs directly sample. - Where motion is low, synthesize intermediate frames using temporal interpolation (GPU shader or CPU optical flow) to preserve smoothness while reducing uploads.

4. Adaptive stride controller - If evictionRate increases (more frames being pushed to storage), increase render sampling density (reduce stride) so visual continuity persists. - If bufferSize increases (more stored frames), we may sample less frequently in time (increase stride) but rely on interpolation to preserve smoothness.

5. Feedback loop & stabilization - The renderer and evictionHook exchange lightweight telemetry to maintain a target display latency and visual smoothness metric. - Use an online PID or simple proportional controller to nudge stride/Markov temperature based on measured jitter / dropped frames.

Sampling & MultiSampler Integration

- MultiSampler is the central sampling engine. Instantiate it inside motion-worker with a per-job timeBudgetMs (use createLightweight for per-frame jobs).
- The worker runs MultiSampler.sample(thumbnailBitmap, options) to produce samplePoints, variance maps, and plugin results.
- Sample manifests include normalized coordinates, weights, sources (wallis/random/vogel), and optional topology tags (prime ends, tubular skeleton indices).
- The main renderer consumes manifests to: prioritize regions for blending, draw strokes or primitives, or seed GPU compute for SDFs or blob generation.
- Keep an explicit contract: manifests are JSON-serializable lightweight objects; heavy binary artifacts remain in Vault/storage.

Hybrid Fresnel → Pascal → Minimizer Pipeline

Overview:

- Purpose: Produce shape primitives and priors (annular outputs, Pascal quads/blobs) from motion-aware inputs that feed the minimizer to produce compact primitives for later generative tasks and rendering.
- Pipeline stages:
 1. Hybrid Fresnel stage: compute annular / wavefront outputs (GPU shader or CPU module) capturing motion-edge distributions and phase-like features.
 2. Pascal quad generation: convert annular outputs into quads and blobs (topologically aware segmentation), producing Pascal blobs as candidates.
 3. Minimizer: optimize blob geometry (energy-based) for compactness, stability and Lipschitz constraints; output simplified SDF or param primitives.
 4. Persistence: Save canonical results (blob meta, SDFs, minimizer provenance) to Vault with standardized metadata.

Integration notes:

- Each stage exposes deterministic parameterization for reproducibility (seeded RNG, versioned transforms).
- The annular outputs are natural inputs for Pascal quad construction — document coordinate transforms clearly.
- The minimizer uses Lipschitz quaternion priors and tubular skeletons as regularizers to keep primitives coherent under motion deformations.
- Telemetry: log provenance including input frame ranges, sample manifests, and parameter sets for reproducibility.

Vault Design & Metadata Schema

Vault responsibilities:

- Long-term artifact retention (calibration, canonical thumbnails, sample-manifests, primitives, SDFs, minimizer outputs).
- Rich metadata indexing for retrieval, provenance and training dataset curation.
- Different retention tiers and pin/unpin semantics (VaultPin vs transient pin in preprocessor).

Suggested metadata fields (per artifact): { key, type, producer, createdAt, frameRange, resolution, sizeBytes, calibrationKey?, srcHash, tags:[motion, scene, primitive], params:{...}, provenance:{jobId, worker, gitSha}, boundingBox, topologySummary }

Search & retrieval:

- Provide query API for time ranges, tags, motionIntensity thresholds, topology signatures.
- Broadcast channel for new artifact notifications to allow downstream consumers to subscribe.

Security & quotas:

- Vault space governed by quota manager in storage.js; automatic eviction policies and recommended soft/hard pin semantics.

Lipschitz Quaternion Motion Maps & Prime-Ends Topology

- Lipschitz Quaternion Motion Maps: a compact descriptor that encodes localized rotational motion and deformation using quaternion fields with Lipschitz regularization. Usage:
 - Consume multi-frame sample patches (thumbnails) to compute quaternion fit per region (worker).
 - Use as a prior in minimizer to prefer rotationally coherent primitives.
 - Provide to renderer or ML pipeline for motion-aware deformation synthesis.

- Prime-Ends Topology & Tubular Skeletons:
 - Prime-Ends based sampling provides robust boundary/end identification for open domains (useful for stroke and blob extraction).
 - Tubular skeletons capture medial axis-like structures for primitives; useful for collision-aware rendering and SDF seeding.
 - Integrate topology plugin into MultiSampler

to annotate sample points with prime-end tags and tubular skeleton indices.

Camera Position & Object Tracking

- Camera tracking: compute relative camera motion (eg. via frame-to-frame feature matching or solvePnP pipelines) - Motion-worker can optionally compute camera pose from thumbnails (cheaper feature matching) and publish cameraTrack objects.
- Use cases: - Integrate cameraTrack into Vault metadata to support later reprojection or multi-view assembly. - Feed camera pose into AmbiAnamorph or scene compositing for object stabilization or generative corrections.
- Note: accurate pose estimation may require higher resolution frames or external calibration; treat computed poses as soft priors with confidence scores.

Preprocessor Enhancements

Additions to preprocessor.worker responsibilities (minimal invasive changes):

- Broadcast camera/track fields when available: include cameraTrack object in thumbnail manifest metadata.
- Emit light telemetry: per-frame captureTime, readW/readH, downsampleScale, calibrationKey, srcHash, frameNumber.
- Optionally generate tiny motion thumbnails (binary motion masks) and store as artifacts for motion-worker to fetch.
- Keep explicit ownership semantics: when preprocessor transfers a thumbnail to main or to motion-worker, mark artifact as transferred and track refcounts (as in current CALIB.refcount pattern).

Telemetry, Tuning & UI

- Provide compact telemetry: framesOffered/processed/dropped, avgProcessingTime, motionIntensity, bufferFillRatio, evictionRate.
- Expose tuning panel collapsed by default; advanced controls include: - Markov temperature, inverse Polya strength, targetDisplayFps, strideMin/Max, interpolationMode, MultiSampler weights and timeBudget.
- Provide exportable telemetry logs for parameter sweeps, and basic visual plots in debug mode.

Implementation Roadmap (Milestones & Tasks)

Milestone 1 — Foundations (safe, minimal risk)

1. Add telemetry & camera/track fields to preprocessor (non-breaking).
2. Expose FrameBuffer metadata getters (if not present) and eviction hook telemetry.
3. Create scaffolding for motion-worker (message API + init); implement simple echo job.

Milestone 2 — Motion-Worker & MultiSampler

1. Implement motion-worker in module worker file using MultiSampler.createLightweight.
2. Worker handles analyzeFrame and samplePoints ops; returns manifests.
3. Integrate MotionDetector.js so worker can optionally use same algorithms (refactor shared utilities into a utils module).
4. Ensure ownership: test ImageBitmap transfers and proper close() calls.

Milestone 3 — Render Policy & Renderer Integration

1. Implement adaptive stride controller and Markov-based selector in MotionPainter main.
2. Integrate sample manifests into render decision pipeline; fallback to MotionDetector for immediate gating.
3. Implement GPU temporal interpolation shader and per-region interpolation driven by motion masks.

Milestone 4 — Fresnel/Pascal/Minimizer & Vault

1. Implement Hybrid Fresnel shader/component to produce annular outputs.
2. Build Pascal quad/blob generator (worker or main) from annular outputs.
3. Implement minimizer module using tubular skeleton and Lipschitz quaternion priors; persist primitives to Vault.
4. Define Vault metadata schema and storage API integration.

Milestone 5 — Advanced features & ML integration

1. Telemetry-driven parameter tuning UI and automated sweeps.
2. Optional supervised/unsupervised training using Vault datasets.

3. Integration of AmbiAnamorph and other uploaded modules for extended capabilities.

APIs & Code Contracts (practical)

- Worker message contracts: use structured JSON with `op` and `jobId` fields; transfer `ImageBitmaps` in `postMessage [bitmap]`.
- Sample manifest schema (example): `{ jobId, timestamp, frameNumber, width, height, samplePoints: [{xNorm,yNorm,weight,source,extra}], // small arrays motionMapSummary: { motionLevel, coverage, peaks: [...] }, topologyTags: { primeEndsCount, tubularSkeletonDescriptor }, metrics: { processingTime, queueUtilization } }`
- Vault artifact meta schema: see Vault Design & Metadata Schema above.

Testing & Validation

- Unit tests: `MultiSampler` outputs, motion-worker message handling, CALIB compute correctness.
- Integration tests: simulate buffer sizes and eviction rates; assert renderer maintains visual smoothness metric (perceptual test or measured frame-to-frame difference and interpolation success).
- Stress tests: high `uploadRate`, varying `bufferSize`, worker queue saturation and backpressure handling.
- Reproducibility: ensure seeded RNG and deterministic pipelines when seeds/params fixed.

Safety, Performance & Resource Management

- Explicit resource ownership: close `ImageBitmaps`, release `OffscreenCanvas` contexts, and control worker concurrency.
- Time-budgeting: `MultiSampler` and heavy topology jobs must respect per-job budgets and `cancelTokens`.
- Memory accounting: expose estimated GPU memory usage per buffer and total MB; use `WebGL2` texture array where supported.
- Backpressure: when motion-worker queue saturates, degrade gracefully (reduce sample density, fallback to `MotionDetector-only` path).

Game Engine & Sandbox Thoughts

- Motion-Painter can evolve into a scene/Entity-Component style sandbox. Essentials of a game engine (rendering, scene graph, physics, input, assets, scripting) overlap with Motion-Painter primitives.
- Collision inference: using motion masks, tubular skeletons, and SDFs you can approximate collisions and resolve rigid/soft interactions in a simplified physics layer.
- For full game engine features, add: scene graph, deterministic fixed-timestep physics module, ECS pattern and networking—these are optional future expansions.

Appendices: File & Module Map

List of key files (existing & planned):

- `src/js/core/WebGLRenderer.js`
- `src/js/core/FrameBuffer.js`
- `src/js/core/MotionDetector.js` (refactor: share utilities with motion-worker)
- `src/js/core/CompositeRenderer.js`
- `src/js/core/MultiSampler.js` (new consumed module)
- `src/js/workers/preprocessor.worker.js` (updated telemetry)
- `src/js/workers/motion.worker.js` (new)
- `src/js/core/FresnelPipeline.js` (new)
- `src/js/core/PascalBlobs.js` (new)
- `src/js/core/Minimizer.js` (new)
- `src/js/core/vault.js` + `storage.js` (Vault integration)
- `src/js/ui/TuningPanel.js` (collapsed control UI)

Closing Notes

This guide provides a detailed, implementable specification to update Motion-Painter into a modular, adaptive, and extensible system. The strategy emphasizes non-breaking, incremental changes, explicit contracts for interprocess communication, telemetry for tuning, and careful resource ownership. Refer to the Milestones & Tasks section when scheduling work. Keep changes small and well-covered by tests.